

Introdução à Programação na Web / Programação na Internet

Inverno de 2022/2023

* Teste Global de Época de Recurso (01 de Fevereiro de 2023) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 1 [7 valores]

Nas seguintes questões, selecione a única opção verdadeira. Uma resposta correta conta a totalidade da cotação da pergunta; uma resposta incorreta desconta 25% da cotação da pergunta ao total do grupo; uma questão não respondida conta 0 valores.

1. Um URI pode ser constituído por:
 - A. *Scheme, port, path, query e header.*
 - B. *Scheme, host, port, path, query e fragment.*
 - C. *Scheme, route, path e fragment.*
 - D. *Scheme, method, host, port, path, query e fragment.*
2. Considere o seguinte URI: `http://some.host/groups/1?prev=/movies/2`
 - A. A *path* é: `/groups/1/movies/2`
 - B. Não existe *query string*
 - C. Não tem *scheme*
 - D. A *query* é: `prev=/movies/2`
3. Um *middleware* express:
 - A. Não pode acrescentar propriedades ao objeto *request*
 - B. Não pode chamar a função *next*
 - C. Não pode interceptar um pedido antes de ele chegar ao *handler*
 - D. Nenhuma das anteriores
4. Em código JavaScript executado pelo browser no âmbito de um documento HTML, a forma correcta de obter a referência para todos os elementos *button*:
 - A. `document.querySelector("#button")`
 - B. `document.querySelectorAll("button")`
 - C. `document.querySelectorAll(".button")`
 - D. `document.querySelectorAll("element=button")`
5. Um pedido HTTP considerado idempotente:
 - A. Pode ter qualquer dos métodos HTTP
 - B. Pode ter apenas o método POST
 - C. Pode ter os métodos DELETE, PUT ou GET
 - D. Pode ter apenas o método GET
6. Na web, um recurso pode ser criado através de um pedido HTTP com:
 - A. Os métodos PUT ou POST
 - B. O método GET
 - C. Qualquer um dos métodos
 - D. Apenas o método POST
7. No contexto do trabalho prático, o header *authorization* foi usado para enviar:
 - A. A *password* do utilizador
 - B. O *username* e a *password* do utilizador
 - C. O *token* associado ao utilizador
 - D. O *username* do utilizador

Introdução à Programação na Web / Programação na Internet

Inverno de 2022/2023

* Teste Global de Época de Recurso (01 de Fevereiro de 2023) - Duração 1h30

NÚMERO _____ NOME _____

8. O método HTTP GET é:
- A. O mais adequado para actualizar um recurso
 - B. O mais adequado para criar um recurso
 - C. Safe e idempotente
 - D. Apenas *safe*
9. O status code 400 é adequado para responder a uma situação em que:
- A. O pedido não pode ser processado devido a um erro do cliente
 - B. O pedido não pode ser processado devido a um erro do servidor
 - C. O pedido foi processado com sucesso
 - D. O servidor está ocupado
10. Numa aplicação express, o código `app.use(express.json())` é necessário para:
- A. Colocar na propriedade *body* do objeto Response o conteúdo do *body* da resposta com Content-Type: `application/json`
 - B. Colocar na propriedade *body* do objeto Request o conteúdo do *body* do pedido com Content-Type: `application/json`
 - C. Quando a propriedade *body* do objeto Response contém um objeto, transformar esse objeto em JSON e adiciona o *header* Content-Type com o valor `application/json` à resposta
 - D. Nenhuma das anteriores
11. A palavra reservada `await` deve ser usada:
- A. Antes de uma expressão que produz uma Promise.
 - B. Numa função que não seja *async*.
 - C. Uma única vez numa função que não seja *async*
 - D. Sempre que se retorna uma Promise
12. Uma aplicação *web* que recebe um pedido de GET para <http://www.cmdb.com/movies/1>, deve:
- A. Responder com status code 200 se o recurso existir e com o status code 404 se o recurso não existir
 - B. Responder com status code 302 se o recurso não existir
 - C. Responder com status code 500 se o recurso não existir
 - D. Responder com status code 201 se o recurso existir e com o status code 404 se o recurso não existir

Introdução à Programação na Web / Programação na Internet

Inverno de 2022/2023

** Teste Global de Época de Recurso (01 de Fevereiro de 2023) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 1 [7 valores]

Nas seguintes questões, selecione a única opção verdadeira. Uma resposta correta conta a totalidade da cotação da pergunta; uma resposta incorreta desconta 25% da cotação da pergunta ao total do grupo; uma questão não respondida conta 0 valores.

1. Considere o seguinte URI: `http://some.host/groups/1?prev=/movies/2`
 - A. Não tem *scheme*
 - B. A *query* é: `prev=/movies/2`
 - C. A *path* é: `/groups/1/movies/2`
 - D. Não existe *query string*
2. Em código JavaScript executado pelo browser no âmbito de um documento HTML, a forma correcta de obter a referência para todos os elementos button:
 - A. `document.querySelectorAll("#button")`
 - B. `document.querySelectorAll("element=button")`
 - C. `document.querySelectorAll("button")`
 - D. `document.querySelectorAll(".button")`
3. Numa aplicação express, o código `app.use(express.json())` é necessário para:
 - A. Colocar na propriedade *body* do objeto Request o conteúdo do *body* do pedido com Content-Type: `application/json`
 - B. Quando a propriedade *body* do objeto Response contém um objeto, transformar esse objeto em JSON e adiciona o *header* Content-Type com o valor `application/json` à resposta
 - C. Colocar na propriedade *body* do objeto Response o conteúdo do *body* da resposta com Content-Type: `application/json`
 - D. Nenhuma das anteriores
4. Na web, um recurso pode ser criado através de um pedido HTTP com:
 - A. Apenas o método POST
 - B. O método GET
 - C. Qualquer um dos métodos
 - D. Os métodos PUT ou POST
5. Um *middleware* express:
 - A. Não pode interceptar um pedido antes de ele chegar ao *handler*
 - B. Não pode chamar a função `next`
 - C. Não pode acrescentar propriedades ao objeto request
 - D. Nenhuma das anteriores
6. Um URI pode ser constituído por:
 - A. *Scheme*, *route*, *path* e *fragment*.
 - B. *Scheme*, *method*, *host*, *port*, *path*, *query* e *fragment*.
 - C. *Scheme*, *port*, *path*, *query* e *header*.
 - D. *Scheme*, *host*, *port*, *path*, *query* e *fragment*.
7. Um pedido HTTP considerado idempotente:
 - A. Pode ter qualquer dos métodos HTTP
 - B. Pode ter apenas o método POST
 - C. Pode ter os métodos DELETE, PUT ou GET
 - D. Pode ter apenas o método GET

Introdução à Programação na Web / Programação na Internet

Inverno de 2022/2023

** Teste Global de Época de Recurso (01 de Fevereiro de 2023) - Duração 1h30

NÚMERO _____ NOME _____

8. O método HTTP GET é:
- A. O mais adequado para actualizar um recurso
 - B. Apenas *safe*
 - C. O mais adequado para criar um recurso
 - D. *Safe* e idempotente
9. Uma aplicação *web* que recebe um pedido de GET para <http://www.cmdb.com/movies/1>, deve:
- A. Responder com status code 302 se o recurso não existir
 - B. Responder com status code 200 se o recurso existir e com o status code 404 se o recurso não existir
 - C. Responder com status code 201 se o recurso existir e com o status code 404 se o recurso não existir
 - D. Responder com status code 500 se o recurso não existir
10. No contexto do trabalho prático, o header *authorization* foi usado para enviar:
- A. O *token* associado ao utilizador
 - B. O *username* do utilizador
 - C. O *username* e a *password* do utilizador
 - D. A *password* do utilizador
11. O status code 400 é adequado para responder a uma situação em que:
- A. O pedido não pode ser processado devido a um erro do servidor
 - B. O servidor está ocupado
 - C. O pedido foi processado com sucesso
 - D. O pedido não pode ser processado devido a um erro do cliente
12. A palavra reservada *await* deve ser usada:
- A. Uma única vez numa função que não seja *async*
 - B. Sempre que se retorna uma *Promise*
 - C. Antes de uma expressão que produz uma *Promise*.
 - D. Numa função que não seja *async*.

Introdução à Programação na Web / Programação na Internet

Inverno de 2022/2023

*** Teste Global de Época de Recurso (01 de Fevereiro de 2023) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 1 [7 valores]

Nas seguintes questões, selecione a única opção verdadeira. Uma resposta correta conta a totalidade da cotação da pergunta; uma resposta incorreta desconta 25% da cotação da pergunta ao total do grupo; uma questão não respondida conta 0 valores.

1. A palavra reservada *await* deve ser usada:
 - A. Numa função que não seja *async*.
 - B. Uma única vez numa função que não seja *async*
 - C. Antes de uma expressão que produz uma *Promise*.
 - D. Sempre que se retorna uma *Promise*
2. Um *middleware* express:
 - A. Não pode interceptar um pedido antes de ele chegar ao *handler*
 - B. Não pode chamar a função *next*
 - C. Não pode acrescentar propriedades ao objeto *request*
 - D. Nenhuma das anteriores
3. Um pedido HTTP considerado idempotente:
 - A. Pode ter apenas o método GET
 - B. Pode ter os métodos DELETE, PUT ou GET
 - C. Pode ter qualquer dos métodos HTTP
 - D. Pode ter apenas o método POST
4. Na web, um recurso pode ser criado através de um pedido HTTP com:
 - A. Os métodos PUT ou POST
 - B. Apenas o método POST
 - C. O método GET
 - D. Qualquer um dos métodos
5. Considere o seguinte URI: `http://some.host/groups/1?prev=/movies/2`
 - A. A *query* é: `prev=/movies/2`
 - B. Não tem *scheme*
 - C. A *path* é: `/groups/1/movies/2`
 - D. Não existe *query string*
6. No contexto do trabalho prático, o header *authorization* foi usado para enviar:
 - A. O *token* associado ao utilizador
 - B. O *username* e a *password* do utilizador
 - C. A *password* do utilizador
 - D. O *username* do utilizador
7. Em código JavaScript executado pelo browser no âmbito de um documento HTML, a forma correcta de obter a referência para todos os elementos *button*:
 - A. `document.querySelectorAll("element=button")`
 - B. `document.querySelectorAll("button")`
 - C. `document.querySelectorAll(".button")`
 - D. `document.querySelectorAll("#button")`

Introdução à Programação na Web / Programação na Internet

Inverno de 2022/2023

*** Teste Global de Época de Recurso (01 de Fevereiro de 2023) - Duração 1h30

NÚMERO	NOME
--------	------

8. O método HTTP GET é:
- A. Apenas *safe*
 - B. *Safe* e idempotente
 - C. O mais adequado para actualizar um recurso
 - D. O mais adequado para criar um recurso
9. O status code 400 é adequado para responder a uma situação em que:
- A. O pedido foi processado com sucesso
 - B. O pedido não pode ser processado devido a um erro do servidor
 - C. O pedido não pode ser processado devido a um erro do cliente
 - D. O servidor está ocupado
10. Numa aplicação express, o código `app.use(express.json())` é necessário para:
- A. Quando a propriedade *body* do objeto Response contém um objeto, transformar esse objeto em JSON e adiciona o *header* Content-Type com o valor `application/json` à resposta
 - B. Colocar na propriedade *body* do objeto Request o conteúdo do *body* do pedido com Content-Type: `application/json`
 - C. Colocar na propriedade *body* do objeto Response o conteúdo do *body* da resposta com Content-Type: `application/json`
 - D. Nenhuma das anteriores
11. Um URI pode ser constituído por:
- A. *Scheme, route, path e fragment.*
 - B. *Scheme, host, port, path, query e fragment.*
 - C. *Scheme, method, host, port, path, query e fragment.*
 - D. *Scheme, port, path, query e header.*
12. Uma aplicação *web* que recebe um pedido de GET para <http://www.cmdb.com/movies/1>, deve:
- A. Responder com status code 302 se o recurso não existir
 - B. Responder com status code 500 se o recurso não existir
 - C. Responder com status code 201 se o recurso existir e com o status code 404 se o recurso não existir
 - D. Responder com status code 200 se o recurso existir e com o status code 404 se o recurso não existir

Introdução à Programação na Web / Programação na Internet

Inverno de 2022/2023

Teste Global de Época de Recurso (01 de Fevereiro de 2023) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 2 [4.5 valores]

1. [1.5] Considere o programa em Node.js apresentado em seguida. Indique justificando, num máximo de 3 linhas, o que é apresentado na consola.

```
let value = 0
console.log("START")
setTimeout(() => { value = 5 }, 1000)
setTimeout(() => { value = 2 }, 3000)
setTimeout(() => { while (value != 2) console.log(`value=${value}. WAITING...`) }, 2000)
console.log("DONE!")
```

2. [3] Considere o código em Node.js apresentado, onde as funções handler1, handler2 e handler3 são handlers e a função mw é um middleware que não pára o processamento do pedido. Assinale, nas colunas correspondentes com Sim ou Não, consoante as funções sejam executadas ou não em cada um dos pedidos (Método e URI path) e justifique cada uma das respostas.

```
const app = express()
app.use("/tasks", mw)
app.get("/tasks/:id", handler1)
app.get("/tasks/top", handler2)
app.get("/tasks", handler3)
```

	Method	URI path	handler1	handler2	handler3	mw
1	GET	/tasks/urgent	S			S
2	GET	/tasks			S	S
3	GET	/tasks				
4	GET	/tasks/1/edit				S

GRUPO 3 [5 valores]

1. [3] Implemente a função filterAsync, que recebe como parâmetros um *array* de strings com URIs e um predicado. Para cada URI do *array*, a função obtém a representação em JSON do respectivo recurso, transformando-a num objeto em JavaScript. Por fim, retorna um *array* apenas com os objetos que respeitem o predicado.

NOTA: Valorizam-se soluções que realizem os pedidos em paralelo.

No exemplo seguinte, considere que a resposta ao pedido para o URI <http://www.example.com/fans/1> contem a representação em JSON {"name":"Maria","club":"SLB"} e a resposta ao pedido para o URI <http://www.example.com/fans/23> contem {"name":"Ricardo","club":"PSG"}. Neste exemplo será apresentado na consola [{ name: 'Maria', club: 'SLB' }].

```
function main(){
  const uris = ["http://www.example.com/fans/1", "http://www.example.com/fans/23"]
  filterAsync(uris, r => r.club == "SLB").then(rs => console.log(rs))
}
```

2. [2]. Considere o seguinte código em JavaScript.

```
1 async function f1() { return 1904 }
2 async function f2(str) { return str + ":SLB" }
3 function f3() { return f1().then(res1 => f2(res1)).then(res2 => console.log(res2)) }
4 f3().then(res3 => console.log(res3))
```

- a. [1] Indique o que é apresentado na consola.
b. [1] Implemente a função f3() usando *await*.

Introdução à Programação na Web / Programação na Internet

Inverno de 2022/2023

Teste Global de Época de Recurso (01 de Fevereiro de 2023) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 4 [3.5 valores]

1. [3,5] Considere os módulos apresentados nas listagens seguintes, que fazem parte de uma aplicação web implementada com o módulo Express em Node.js.
- [1.0] Complete a implementação do módulo `middleware.mjs` que disponibiliza a funcionalidade de contagem do número de pedidos feitos ao servidor por método HTTP. Além disso, coloca na propriedade `counters` do objeto `Request` um objeto com os contadores por método.
 - [2.5] Complete a implementação do módulo `server.mjs`, que expõe um recurso com a informação com o número de pedidos efetuados, que recebe na query o método para o qual se pretende obter o contador. O acesso a este recurso não altera qualquer contagem.

middleware.mjs			
1	const counters = { GET : 0, POST : 0, PUT : 0, DELETE : 0 }		
2			
3	function middlewareConstructor(pathToExclude){		
4	function middleware(req, rsp, next){		
5	if(req.path !== pathToExclude){		
6	++counters[req.method]		
7	}		
8	_____ //1 40%		
9	_____ //2 30%		
10	}		
11	return middleware		
12	}		
13	_____ // 3 30%		

server.mjs			
1	import express from "express";		
2	import middlewareConstructor from "./middleware.mjs"		
3			
4	const app = express()		
5			
6	app.use(_____) //1 20%		
7	app.put("/puthandler",puthandler)		
8	app.post("/posthandler",posthandler)		
9	app.get("/gethandler",gethandler)		
10	app.get("/methodCounter", _____) //2 15%		
11			
12	app.listen(9000, ()=> console.log("Server listening in port 9000"))		
13			
14	function puthandler(req,rsp){rsp.json({message : "put handler"})}		
15	function posthandler(req,rsp){rsp.json({message : "post handler"})}		
16	function gethandler(req,rsp){rsp.json({message : "get handler"})}		
17			
18	function methodCounter(req,resp){		
19	if(!req.query.method)		
20	return resp.status(_____).json({message:"Missing Method"}) //3 20%		
21	resp.json({		
22	method : _____, //4 15%		
23	counter : _____ //5 30%		
24	})		
25	}		

Os docentes,
Luís Falcão e Filipe Freitas